Dread, et al.

mkPIVM: Position-Independent Virtual Machines

Per-Seed Polymorphic Virtualization of Position-Independent Shellcode for Windows x86 and x64

Abstract

We present both the tool, **mkPIVM**, and the introduction of the concept of position-independent virtual machines: per-instance polymorphic process VMs whose output blob is itself position-independent code, suitable for any loader that accepts native shellcode. mkPIVM generates functionally identical, polymorphic position-independent virtual machines for the x86 and x64 instruction set architectures with an acceptable size and performance differential relative to the original input position-independent shellcode. The tool lifts arbitrary input shellcode to a custom intermediate representation, runs IR-level obfuscation passes including dead-instruction injection and opaque predicates via block splitting[1], encodes the result into per-seed virtual-machine bytecode under one of four stream cipher families, and emits a position-independent blob that behaves identically to the original while varying substantially at the byte level across builds. Per-seed variance spans the cipher family, the VMState register slot permutation, the opcode-to-handler mapping, the dispatcher topology, the prologue self-locate strategy, the per-handler temporary register layout, the junk gadget density, and the runtime nonce that masks the in-memory handler table per process. The tool exposes five operating modes: full virtualization, packer mode for shellcode too exotic or too large to lift, hybrid range mode that lifts only operator-chosen byte ranges, a stacked composition that pack-wraps a hybrid blob, and a PE detour mode that embeds a pre-built blob into an arbitrary Portable Executable[2] and patches a five-byte jump at an operator-chosen relative virtual address so that the virtual machine only fires when the host process reaches that point. The architecture is designed to accommodate the breadth of payload structures observed in modern offensive tooling, ranging from simple linear stagers to multi-stage beacons that perform their own PEB walks, API hash resolution, and direct or indirect syscall invocation[3].

1. Introduction

The Static Signature Problem in Offensive PIC

Position-independent code, in the form of self-contained executable byte streams that resolve their own imports and address their own data through relative addressing, is the dominant payload format in modern offensive operations on Windows. Cobalt Strike beacons, Metasploit meterpreter and reverse TCP stages, and the bulk of public red team tooling all ship raw byte arrays meant to be copied into an executable allocation and called. The format is convenient: any loader that can write memory and create a thread can run the payload, and the payload itself carries no metadata that would otherwise constrain where or how it executes.

That convenience has a cost. Two builds of the same Metasploit payload produce identical bytes byte for byte. Two builds of the same Cobalt Strike beacon, given the same malleable profile, do likewise. The static byte image is the signature, and the signature is the same across copies. Antivirus and endpoint detection vendors index these byte sequences directly, with overlapping n-gram heuristics catching the variants that simple key-rotation encoders attempt to introduce. The arms race in the wild consists of an attacker rotating an XOR key, prepending a fresh stub, or stitching together API-resolution gadgets in a slightly different order, and a vendor pushing a fresh signature within a working day.

A useful obfuscation system for this class of payload has to do something more aggressive than rotate a key. It has to change the byte image enough that two builds from the same input share almost no contiguous bytes, while keeping the runtime semantics exact. The output also has to remain position-independent: it must run wherever the original would have run, with no extra loader-side scaffolding, because the loader side of the operation is already a heavily constrained context.

Why Existing Packers and Protectors are a Poor Fit

Commercial code virtualizers such as VMProtect, Themida, and Code Virtualizer solve a related problem for Windows PE binaries. They lift selected functions to a custom virtual machine, encrypt sections, and emit a protected executable. Their inputs are linked PEs with an entry point, an import table, optionally TLS callbacks, and a known section layout. Their outputs are also PEs, often with additional dependencies and runtime resolver code that assumes a fully formed loader environment.

Raw shellcode is none of those things. It has no PE header, no imports section, no relocations beyond what the shellcode itself performs at runtime through PEB walks and hash-resolved API lookups, and no place to put a runtime resolver DLL. Applying a PE-oriented protector to shellcode, where it can be done at all, requires wrapping the shellcode in a synthetic PE, applying the protector, then extracting the relevant runtime back out, and even then the resulting bytes typically assume loader behavior that the operator's runtime context cannot provide.

The other category of available tooling is shellcode encoders and crypters. Encoders, single-stage XOR wrappers, Donut-style position-independent loaders, and AES-decrypting shim stubs all sit in this category. They provide a per-build encryption layer, which defeats static byte matches on the encrypted body, but the decrypt stub itself is small and structurally identical across builds. The stub becomes the next signature. Worse, none of these systems virtualize the underlying instructions: once decrypted, the original byte

image sits in memory verbatim and is recoverable by any tool that watches the decrypt buffer.

What is missing is a tool that takes raw shellcode in, produces raw shellcode out, virtualizes the original instructions into per-build virtual machine bytecode, and varies the virtual machine itself across builds so that even the wrapper carries no fixed signature.

Contributions

We make three primary contributions.

First, we introduce the concept of position-independent virtual machines: process virtual machines whose entire image, including the dispatcher, handler table, bytecode, and encrypted data island, is itself position-independent code that can be invoked from any loader that accepts shellcode. The construct preserves the deployment surface of native shellcode while replacing the static byte image with a per-instance, per-seed virtual machine.

Second, we present mkPIVM, an implementation of this concept for Windows x86 and x64. The tool lifts arbitrary input shellcode to a custom intermediate representation, applies IR-level obfuscation passes including dead-instruction injection and opaque predicates via block splitting[1], encodes the result into per-seed virtual machine bytecode under one of four stream cipher families, and emits a position-independent blob. Eight axes of per-seed variance vary independently across builds: the cipher family, the register slot permutation across the virtual machine state, the opcode-to-handler mapping, the dispatcher topology, the prologue self-locate strategy, the per-handler temporary register layout, the junk gadget density, and a runtime nonce that masks the in-memory handler table per process.

Third, we describe five operating modes that together cover the breadth of payload deployment shapes encountered in modern offensive tooling: full virtualization, packer mode for payloads too exotic to lift in full, hybrid range mode for selective virtualization of operator-chosen byte ranges, a stacked composition of the two, and a PE detour mode that embeds a pre-built blob into an arbitrary Portable Executable[2] and patches a five-byte jump at an operator-chosen relative virtual address. The architecture is designed to accommodate payloads ranging from simple linear stagers to multi-stage beacons that perform their own PEB walks, API hash resolution, and direct or indirect syscall invocation.

The remainder of the paper is organized as follows. Section 2 reviews relevant background on position-independent code, virtualizer-protectors, and polymorphic engines, and states the threat model. Section 3 describes the system pipeline and the five operating modes at a high level. Section 4 details the lifter and intermediate representation. Section 5 covers the per-seed polymorphism axes. Section 6 describes the IR obfuscation passes and operand encoding. Section 7 covers the runtime virtual machine architecture. Section 8 returns to the operating modes in detail. Section 9 discusses limitations, deliberate tradeoffs, and positioning against prior art. Section 10 concludes.

2. Background and Threat Model

PIC, Virtualizer-Protectors, Polymorphic Engines

Three lines of prior art together define the design space that mkPIVM occupies: position-independent code as a payload format, code virtualization as an obfuscation technique, and polymorphic transformation as a method of defeating static signatures.

Position-independent code. A position-independent code blob is a self-contained byte sequence that executes correctly no matter where in memory it is placed, without requiring a base relocation table or any loader-side fixups. The property is achieved through disciplined use of relative addressing. On x64, RIP-relative addressing is part of the instruction encoding for most memory operands, which makes the construction of position-independent data references straightforward. The x86 architecture lacks an EIP-relative addressing mode for general-purpose memory operands, so a position-independent x86 payload conventionally locates itself with a short call to the instruction following it followed by a pop, which leaves the address of that pop in a register and gives the payload a known anchor from which to compute its own internal offsets[3]. Imports are resolved at runtime through walks of the Process Environment Block, hash-comparing the names of exported symbols against precomputed digests rather than reading an import table that does not exist. The format is well established in offensive operations: stagers, beacons, and post-exploitation payloads in modern adversary-simulation frameworks including Cobalt Strike, Metasploit, and Sliver are distributed as raw byte arrays meant to be copied into an executable allocation and called.

Virtualizer-protectors. Code virtualization is a software-obfuscation technique in which selected functions of a target program are lifted to a custom bytecode and interpreted at runtime by an embedded virtual machine. The encoding of the bytecode and the structure of the virtual machine are chosen to be opaque to static disassembly: native instructions are replaced with operations in an unfamiliar instruction set whose semantics must be inferred from the dispatch loop. Commercial implementations including VMProtect, Themida, and Code Virtualizer apply this technique to Windows Portable Executable binaries[2], typically alongside section encryption, debugger detection, and licensing checks. These tools assume their input is a linked executable with imports, sections, and loader metadata, and their output is itself a PE that depends on a fully formed loader environment. Applying them to raw shellcode requires substantial scaffolding, and even when the scaffolding succeeds the resulting bytes typically retain runtime assumptions, such as preserved imports and a writable data section, that the operator's deployment context cannot meet.

Polymorphic engines. Polymorphism in this domain is the property that successive builds of the same payload produce different byte sequences while preserving runtime semantics. The technique substantially predates contemporary shellcode work. The first widely studied polymorphic virus, the 1260 virus written by Mark Washburn in 1990, used sliding XOR keys together with inserted junk instructions and permuted decryptor segments to produce a different byte image on each replication. The Mutation Engine released by Dark Avenger in 1991 packaged similar transformations as a reusable object that could be linked into any simple virus to give it a polymorphic shell[4]. A taxonomy of obfuscating transformations covering dead-code insertion, opaque predicates, and control-flow restructuring was laid out by Collberg, Thomborson, and Low in 1997[1] and remains the canonical academic reference for the technique class. Public shellcode polymorphism in current red-team tooling is dominated by msfvenom-style encoders such as **shikata_ga_nai**, which apply per-build XOR keys with additive feedback so that the encrypted body differs between builds, prefixed by a small decoder stub whose instructions are dynamically reordered and register-substituted[5]. Donut and similar position-independent loaders

extend the same general approach with stronger ciphers and richer wrapper logic. In every case the wrapper retains a recognizable structural fingerprint across builds, providing detection vendors with a stable signature target, and the original instructions are restored verbatim in memory before they execute.

The gap that mkPIVM addresses sits at the intersection of these three lines. The construct combines a polymorphic obfuscation engine, a code-virtualization layer, and the requirement that the entire output, wrapper and bytecode together, be position-independent code suitable for deployment in any context that accepts native shellcode.

Adversary Model and Non-Goals

mkPIVM is designed against four categories of adversary.

Static signature scanners. Antivirus engines and on-disk detection products match byte sequences in a candidate file against signature databases of known malicious payloads. The matchers range in sophistication from exact n-gram lookups to fuzzy hashes that tolerate small edits, but their input is the static byte image of the file. The threat model assumes that for any given input shellcode, the static byte image of any single build of that input is or will become a signature, and the design objective is to ensure that no two builds share enough contiguous bytes for one signature to match the other.

In-memory scanners. Endpoint detection products and post-exploitation memory scanners walk the address space of running processes and apply signature matching to memory pages, often with the same signature database used for on-disk scanning. The threat model treats memory scans as a cross-process exercise: two processes running the same blob should not produce identical byte images at corresponding offsets in their respective address spaces, so that a scanner cannot pivot from a sample of one running process to a generic signature for the other. A runtime-derived nonce that masks the in-memory handler table per process is the architectural mechanism intended to satisfy this requirement.

Behavioral and sandbox analysis. Some detection products execute candidate payloads in instrumented sandboxes and inspect runtime behavior, including the syscall sequences a payload performs and the API names it resolves. The threat model treats such analyzers as orthogonal to the static and memory-scan defenses: the architecture is, by construction, transparent to whatever behavior the inner payload performs once it executes, since the virtual machine restores the architectural state of the original shellcode before yielding control to native code. Defeating behavioral analyzers is therefore the responsibility of the inner payload, not of the wrapper.

Manual reverse engineering. An analyst examining a captured sample with a static or dynamic disassembler should be required to recover the per-instance virtual machine semantics before the lifted bytecode can be read, and should be unable to reuse that recovery effort across other builds because the cipher family, register layout, opcode mapping, dispatcher topology, and other per-seed knobs differ between any two seeds. The architectural goal is to make the work performed against one captured sample non-transferable to any other sample of the same input.

Several adversary capabilities are explicitly out of scope.

Anti-debug and anti-virtualization behaviors are not part of mkPIVM. The output blob does not check for the presence of debuggers, does not detect virtual environments, and does not alter its behavior under instrumentation. The reasoning is twofold: such checks are

themselves strong signature targets when present, and they offer at best a temporary advantage against a determined analyst.

Userland API hook bypass is delegated to the inner payload. mkPIVM does not unhook ntdll, does not perform syscall-stub resolution on behalf of the payload, and does not interfere with the payload's own choice of direct, indirect, or through-API invocation. The architecture is transparent to whichever invocation style the inner shellcode uses.

Kernel-mode defenses, including kernel callbacks, Early Launch Anti-Malware enforcement, and PatchGuard interactions, are out of scope. mkPIVM operates entirely in userland and treats the kernel surface as a fixed boundary.

Cryptographically secure obfuscation, in the sense of indistinguishability obfuscation or formally proven resistance against polynomial-time adversaries, is not a design goal. The ciphers used to encrypt the bytecode and handler table are stream constructions chosen for byte-level diffusion against signature matching, not for resistance against an adversary with extended observational access to the running process.

Authenticode signing and other code-signing infrastructure interactions are out of scope. The output blob is unsigned, and any host PE patched by the detour mode loses its embedded signature and its checksum, by design.

3. System Overview

Pipeline: Raw PIC In, Polymorphic PIC Out

The mkPIVM build pipeline accepts a raw shellcode byte array together with an architecture flag and a 64-bit seed, and produces a raw output byte array that is itself position-independent code with the runtime semantics of the input. The seed is the only source of per-build variance; the architecture flag is independent of the seed. The pipeline is divided into seven stages.

Stage 1, ingest. Input bytes are read into a buffer alongside the operator-supplied flags. Input-format converters accept conventional encodings including raw binary, hexstreams, and C-array escape sequences.

Stage 2, polymorphism setup. A VMConfig object is derived deterministically from the seed. The configuration holds the choice of cipher family, the register slot permutation across the VMState, the opcode-to-handler mapping, the dispatcher topology, the prologue self-locate strategy, the per-handler temporary-register layouts, the junk gadget density, and the initial cipher state.

Stage 3, lifting. In default mode the input is disassembled instruction by instruction over a recursive-descent control flow graph builder, and each instruction is lowered to a custom intermediate representation by a per-mnemonic lifter routine. In packer mode the lifter is bypassed entirely, and the input is treated as opaque data accompanied by a single synthetic IR instruction that performs the native escape. In hybrid range mode only the byte ranges specified on the command line are lifted; the rest of the input remains as native bytes.

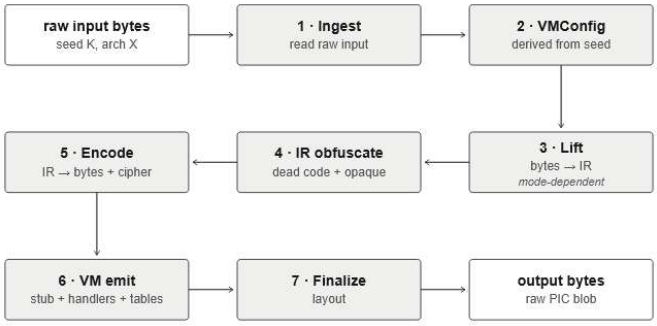
Stage 4, IR obfuscation. Two IR-level passes run between lifting and bytecode encoding. The first inserts dead-IR instructions, drawn from a class of operations that produce no architecturally observable side effect, into random gaps between live instructions. The second introduces opaque predicates by splitting blocks and inserting always-false conditional branches whose targets are syntactically valid but never reached at runtime. Both transformations are drawn from the standard

taxonomy of obfuscating transformations[1]. Each pass uses a sub-derived random number generator so that toggling its density does not perturb the rest of the codegen.

Stage 5, encoding. A codec table walks the IR program and emits bytecode bytes through per-seed encoder routines, one per IR opcode family. Operand encodings are obfuscated at this stage: immediate constants are split into four-component expressions that recombine at runtime to the original value, and each byte emitted is XORed with the running cipher state. The bytecode is therefore encrypted in place by construction rather than as a separate encryption pass.

Stage 6, VM emission. The codegen module assembles the wrapper virtual machine. It emits the prologue, the state-initialization body, the dispatcher tail, the handler bodies, the inverse substitution box for the chosen cipher family when applicable, and the exit handler. The handler table, mapping each opcode byte to the offset of its handler relative to the dispatch base, is laid down as a fixed-size array of 256 four-byte entries. The handler table, block table, and data island are encrypted with the same cipher family that was used during bytecode emission.

Stage 7, finalize. The artifacts are concatenated in a fixed layout consisting of the VM stub, the inverse substitution box, the bytecode, the data island, and the block table. Internal references resolve through a fixup table that the codegen accumulates during emission. The result is a single contiguous byte array suitable for any deployment context that accepts native shellcode.



The Five Modes and the Seed as Sole Source of Variance

mkPIVM exposes five operating modes that share the same virtual machine construction but vary in how much of the input is lifted and how the output is packaged.

Default mode lifts the entire input. The output is a single self-contained blob whose execution begins in the VM prologue and remains inside the dispatch loop except where the lifted code explicitly transfers control to native execution, for example to invoke a resolved Windows API.

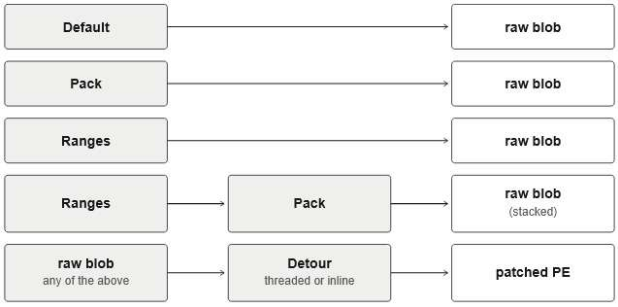
Packer mode skips the lifter and treats the input as encrypted data. The IR consists of a single synthetic native-escape instruction whose effect is to decrypt the data island on first entry and transfer control to the now-plaintext shellcode. The mode preserves the per-seed virtual machine polymorphism on the wrapper while leaving the inner shellcode native after decrypt.

Hybrid range mode lifts only the byte ranges supplied on the command line. Each range start is patched with a five-byte relative jump to a per-range VM entry stub, and the body of the range is filled with int3 so that any control transfer that enters a mid-range byte from outside terminates loudly rather than silently. Lifted code that branches out of its range emerges back into native execution through the native-escape paths.

Stacked mode is the lexical composition of packer mode wrapping hybrid range mode. The

inner blob is a hybrid range output, and the outer wrapper is a packer-mode virtual machine. The two virtual machines are independent: they hold distinct VMState frames, run distinct cipher schedules, and can be invoked with distinct seeds when the operator chooses to vary them.

PE detour mode is the only mode whose output is not a raw blob. It accepts a pre-built VM blob together with a Portable Executable target, patches a five-byte relative jump at an operator-chosen relative virtual address into a freshly appended wrapper that calls into the blob, and writes a modified PE back out. The mode comes in two sub-modes: a threaded variant that runs the blob in a worker thread via a kernel32!CreateThread call resolved from the host's import address table, and an inline variant that calls the blob synchronously on the host thread.



The seed is the only source of variance across builds of the same input. Two invocations of mkPIVM with identical inputs, identical mode flags, and an identical seed produce byte-identical output, which is essential for reproducible builds and for the operator workflow in which a known-good seed is replayed across testing and deployment. Two invocations with different seeds produce outputs that differ in every architectural axis listed in stage 2 of the pipeline. The seed value is supplied by the operator on the command line; the architecture flag, the mode flags, and any range or detour-target arguments are orthogonal to it.

4. Lifting x86 and x64 to a Custom IR

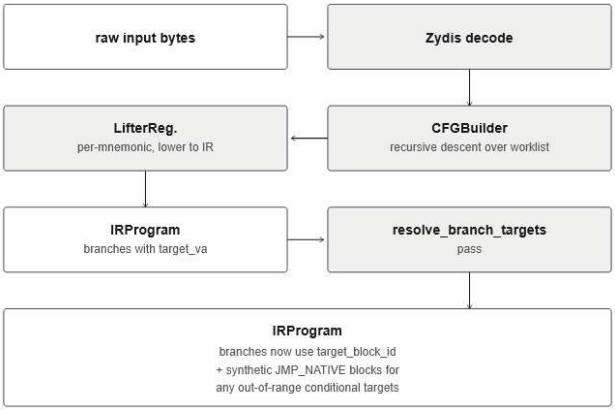
The lifter is responsible for transforming the input shellcode's native machine code into a sequence of instructions in a custom intermediate representation that the virtualization stack later encodes as bytecode. The lifter runs in default mode and in hybrid range mode; packer mode bypasses it entirely. This section describes the two-stage lifting pipeline: first, control flow graph recovery together with per-mnemonic instruction lowering; second, post-lift branch target resolution including synthesis for branches whose targets fall outside the lifted region.

CFG Recovery and the Per-Mnemonic Lifter Registry

CFG recovery. The lifter consumes the input shellcode through Zydis[6], a permissively-licensed x86 and x64 instruction decoder. Decoding proceeds via recursive descent disassembly[7]: a worklist is seeded with the input entry point, the instruction at the head of the worklist is decoded, and any control flow successors of that instruction are pushed back onto the worklist provided they fall within the lifted span. Successors are computed from the decoded instruction's category and operands. Non-branch instructions and call instructions yield a single linear successor at the byte immediately following the instruction. Conditional branches yield both the explicit branch target and the fall-through. Unconditional branches yield only the explicit branch target. Indirect transfers and returns yield no recursive descent successors at all, since the resolved target is not statically known.

Block boundaries. The CFG groups instructions into basic blocks. A block ends at any instruction whose effect on control flow is not a simple linear advance: conditional branches,

unconditional branches, calls, returns, and indirect transfers all terminate a block. A block also ends defensively at any byte address that is the target of some branch from elsewhere in the graph, because such a target must be the start of a fresh block even if the byte that precedes it does not terminate the previous one. The result is a directed graph whose nodes hold ordered lists of decoded instructions together with metadata such as the block's starting virtual address, and whose edges carry the resolved control flow relationships between blocks.



Lifter registry. With the CFG in hand, the lifter walks each block in order and lowers each decoded instruction to one or more IR instructions through a dispatch table keyed by the source mnemonic. The dispatch table is a registry of small lifter routines, one per mnemonic family, each responsible for emitting the IR instructions that semantically realize the source instruction's effect on architectural state. The registry pattern allows new mnemonics to be added by registering a new lifter routine, and it isolates the IR-emission complexity of any given mnemonic family from the others. Each routine receives the decoded instruction together with handles to the IR program under construction and to a register-to-slot mapping table.

Routines emit IR operations drawn from a small set of opcodes: LOAD for memory reads, STORE for memory writes, IMM for immediate-to-temporary loads, LEA for address arithmetic, the arithmetic and bitwise opcodes for native ALU instructions, shift opcodes for SHL, SHR, SAR, ROL, and ROR, comparison opcodes that feed flag-aware consumers, BR_CC for conditional branches, BR for unconditional branches, CALL_VM for intra-blob calls, CALL_NATIVE for calls whose targets leave the lifted region, RET_VM as the call-return terminator, and JMP_NATIVE as the unconditional native-escape terminator. Operand width is encoded as a per-operand attribute rather than by opcode multiplication, which keeps the opcode count manageable and confines width-specific behaviour to the codecs and handlers rather than spreading it across the IR.

A NopLifter alias group collects instructions that have no architectural effect within the lifted region: traditional NOP, ENDBR64 and ENDBR32 control-flow integrity hints, PAUSE, and INT3 when it appears as alignment padding. Each is folded to a single IR NOP. Unrecognized opcodes are dispatched to a default case that also emits NOP, with a logging hook so that operator-supplied shellcode containing rare or unsupported mnemonics surfaces as a warning rather than as a hard build failure.

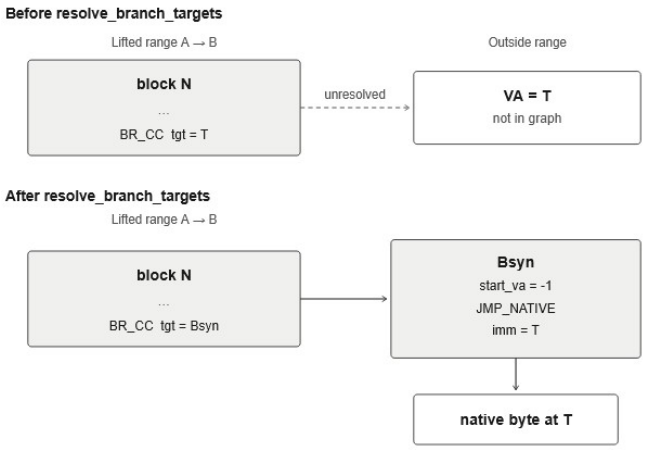
Branch Resolution and Out-of-Range JCC Synthesis

Branch resolution. Lifters emit branches with their target stored as a virtual address rather than as a block reference, because the target block may not yet exist at the moment the source branch is lifted, especially under forward control flow inside a function whose blocks are produced in worklist order rather than in CFG order. After all blocks have been lifted, a post-lift pass named resolve_branch_targets walks every BR, BR_CC, CALL_VM, and LOOP_DEC instruction in the program, looks up the target virtual address in the CFG's

address-to-block-identifier map, and rewrites the target field from a virtual address to a block identifier. Downstream IR encoding uses block identifiers rather than virtual addresses, so this rewrite is required before the bytecode encoder can run.

Out-of-range jcc synthesis. A complication arises in hybrid range mode, where only operator-chosen byte ranges of the input are lifted. Conditional branches inside a lifted range may target instructions that fall outside that range. The simplest representation, lowering such a branch to a JMP_NATIVE-style primitive at lift time, is rejected for two reasons. First, the per-mnemonic lifter routines that emit BR_CC for x86 jcc instructions do not have ambient knowledge of the range boundaries; they treat every conditional uniformly as if it stays inside the lifted region. Second, the BR_CC handler in the runtime is structured around a target block identifier, not an absolute address, so emitting a different opcode at lift time would require teaching the lifter about range mode and adding a separate handler for the out-of-range case.

Instead, the resolve_branch_targets pass synthesizes a fresh, single-instruction block for every conditional branch whose target lies outside the lifted region. The synthetic block holds one IR instruction, a JMP_NATIVE whose immediate operand is the absolute target virtual address, and a sentinel start_va that marks the block as not corresponding to any real input byte. The original BR_CC's target is rewritten to point at this synthetic block. The mechanism preserves the in-VM-only invariant of BR_CC while still allowing conditional control flow to escape the lifted region: the conditional check happens inside the VM, and only on the taken path does control leave through the synthetic JMP_NATIVE.



LOOP_DEC, which lowers x86 LOOP and similar instructions that combine a decrement with a conditional branch, uses the same synthesis path: when its target falls outside the lifted region, the synthesized JMP_NATIVE block becomes the LOOP_DEC's target, and the runtime behavior matches a native LOOP that exits the range on its taken edge.

Unconditional BR and CALL_VM instructions whose targets fall outside the range are handled by direct rewriting rather than synthesis. BR rewrites to JMP_NATIVE with the target virtual address as its immediate, and CALL_VM rewrites to CALL_NATIVE, because the cipher-state reset and dispatcher-yield work that those VM-internal terminators perform is also performed by their native-escape counterparts. The synthesis path is reserved for BR_CC and LOOP_DEC, where the per-path branching structure requires that a block-identifier target exist even though the taken edge crosses into native execution.

5. Per-Seed Polymorphism

Per-seed polymorphism is the property that two builds of the same input produce byte images that share almost no contiguous bytes while preserving exact runtime semantics. mkPIVM achieves this through several independent axes of variance, each driven by the per-build VMConfig that is deterministically derived from

the operator-supplied seed. The axes vary independently so that a static signature anchored on any single axis fails to match a build whose other axes have rotated. This section describes the axes in two groups: first the cipher and the register state layout, then the opcode mapping, the dispatcher topology, and the per-handler internals.

Cipher Families and Register Slot Permutation

Cipher families. The bytecode, the block table, the handler table, and the data island are stored at rest under a per-byte stream cipher chosen from one of four families. The choice is made deterministically from the seed and is the same for all four regions in a given build. The runtime fetch operation that produces the next decoded byte is parameterized over this choice at codegen time, so a build emits only the machine code for its chosen family rather than a runtime switch over all four.

The ARX family follows the add-rotate-xor construction popularized by Salsa20 and ChaCha20[8]. The cipher state evolves under a fixed sequence of 32-bit additions, bitwise rotations, and exclusive-ors, and each byte of plaintext is xored against the low byte of the evolved state to produce ciphertext. Decryption is identical to encryption because the construction is a stream cipher.

The LcgSub family uses a linear congruential generator[9] over the cipher state, with the low byte of each generated value taken as the keystream byte. The multiplier and increment of the LCG are drawn per seed and remain constant throughout a build, so the keystream is deterministic for a given seed.

The SBoxAdd family advances the cipher state through a per-seed inverse substitution box, where each state byte is substituted through a 256-entry table and then summed with a running offset. The inverse substitution box is laid down in the emitted blob at a fixed offset within the VMState's cipher scratch region and is itself encrypted at rest under the chosen cipher family in a self-referential construction that bootstraps from the build-time initial state cipher_init.

The FeistelByte family applies a small Feistel network[10] over byte pairs of the cipher state. Each round splits the state into halves, applies a key-mixed round function to one half, and exclusive-ors the result into the other before the halves swap.

All four families share the same external interface: an inline fetch operation that reads one ciphertext byte from the current bytecode pointer, advances the cipher state, and returns the plaintext to the calling handler. The dispatcher does not know which family is in use; the choice is baked into the emitted machine code through codegen-time switches. The encoder side of the cipher matches the decoder by construction. The matching property is load-bearing: any mismatch between encoded operand byte counts and the byte counts that handlers fetch causes the stream cipher to drift, and downstream handlers dispatch on bytes that decrypt to opcodes the encoder never emitted.

A cipher_reset operation, callable from handler bodies, restores the cipher state to its build-time initial value cipher_init. The reset fires unconditionally on entry to every new basic block, because basic blocks are encoded with the cipher state at cipher_init on entry: the encoder resets between blocks, and the decoder must match the same convention. The reset also fires on the not-taken edge of BR_CC and after CALL_VM returns, because both edges of a conditional branch and the return edge of a call terminate at fresh blocks.

Register slot permutation. The VMState is a contiguous array of fixed-width slots that hold the virtualized values of the architectural

general-purpose registers, four temporary registers used by the lifter, the cipher state, and the per-process runtime nonce. The IR refers to registers by a logical slot identifier such as AX, CX, BP, or Tmp0; the codegen translates each logical identifier through a seed-derived permutation into a numeric slot index. The same logical register lands at different VMState byte offsets in different builds.

The permutation is bijective, deterministic from the seed, and applied uniformly across all references to a given logical register so that the IR remains semantically valid after translation. A static signature that depends on the offset of any particular logical register within the VMState therefore fails to generalize across seeds.

Opcode Mapping, Dispatcher Topology, and Handler Internals

Opcode-to-handler mapping. The IR opcode set contains a small set of opcode families covering arithmetic, bitwise operations, shifts, comparisons, memory load and store, branches, calls, native escapes, and the immediate-and-address primitives. Each family is assigned a numeric opcode byte per seed through a permutation of the available opcode space. The 256-entry handler table at the head of the dispatch region maps each opcode byte to the offset of its handler relative to the dispatch base. The encoded bytecode therefore references the per-seed opcode bytes, and a static scanner that has memorized the opcode byte for ADD in one build finds no occurrence of that byte in a different build, because the byte now means a different operation.

Dispatcher topology. Two dispatcher shapes are emitted, chosen per seed. The threaded shape inlines the fetch-and-dispatch sequence at the end of every handler, so that each handler ends with a fetch of the next opcode byte through the stream cipher, an exclusive-or with the runtime nonce to unmask the handler table entry, and an indirect jump through the resolved handler address. The central shape emits the fetch-and-dispatch sequence exactly once, in a dispatcher tail block, and each handler reaches the next instruction by an unconditional jump to that tail block. The two shapes are operationally equivalent but produce different static layouts and different control flow graphs to any static analysis tool that walks the wrapper.

Handler internals. Each handler is a small piece of architecturally idiomatic code that fetches its operands from the bytecode through the cipher, performs the semantic operation on the slot indicated by the operand, advances the bytecode pointer, and proceeds to the next instruction. Handlers internally use a small pool of host volatile registers for their working set, and the assignment of those registers to logical roles within the handler is permuted per seed. The BR_CC handler, the most complex of the family, has six distinct temporary roles spanning the condition evaluation, the target block table lookup, the cipher state reset, and the dispatcher yield; each role is bound to a host register drawn from a per-seed permutation pool that excludes the registers whose roles are fixed by external constraints, such as the cipher state register and the dispatch base register. The Store handler permutes its temporaries on the same principle.

Prologue self-locate strategy. Position-independent code that needs to refer to its own internal data must first locate itself in memory. The prologue uses one of three strategies, chosen per seed. The call-and-pop strategy issues a short call to the instruction immediately following it and then pops the return address from the host stack, yielding an anchor register that points to the byte after the call. The RIP-relative lea strategy, available only on x64, loads the address of a known label through an RIP-relative effective address computation. The jmp-call shuffle strategy combines a short forward

jump with a call back to an earlier anchor, producing a control flow shape that is less visually obvious in a disassembly listing than either of the simpler strategies.

Junk gadget density. Between handlers the codegen optionally emits zero to three short gadgets of architecturally inert instructions, chosen from a small library of byte sequences that consume bytes without changing any flag or register on which the handlers depend. Junk density contributes to byte-level diversity in two ways. First, the gadget content itself differs between builds. Second, and more importantly, the absolute offset of every handler within the dispatch region shifts according to how many junk bytes precede it, so the entries of the handler table point at different absolute locations across builds even when the handler bodies themselves would otherwise be byte-identical.

6. IR Obfuscation and Operand Encoding

This section describes two transformations that run between the lifter and the runtime virtual machine and that together account for much of the per-build diversity at the bytecode and operand level: IR-level obfuscation passes that introduce structural and semantic noise into the intermediate representation, and a multi-component immediate encoding that diversifies the byte image of literal constants. A separate but related invariant, the byte-count parity between the encoder and the runtime handler for each codec, closes the section as the load-bearing correctness property without which the stream cipher silently desynchronizes.

Dead-IR Injection and Opaque Predicates via Block Splitting

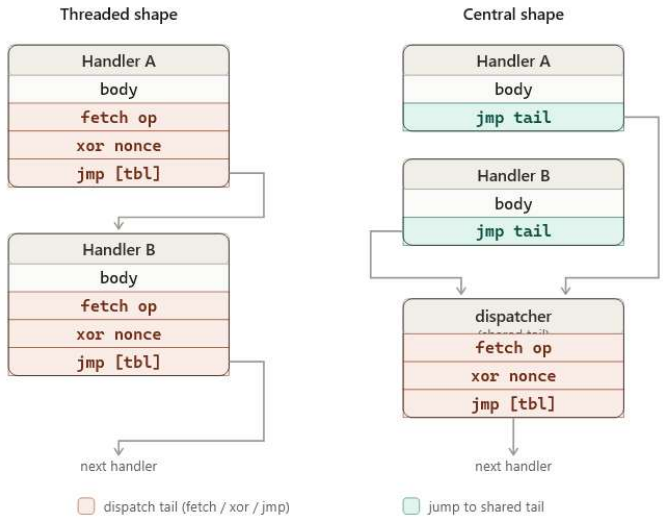
Both passes operate on the IR program produced by the lifter, between the resolve_branch_targets pass described in section 4 and the bytecode encoding step described later in this section. The transformations they implement, dead-code injection and opaque predicate insertion, are drawn from the standard taxonomy of obfuscating transformations[1].

Sub-RNG isolation. Each pass derives a random number generator distinct from the master configuration RNG by calling master.derive on a pass-specific tag string. The master RNG state is left untouched by the obfuscation passes. The isolation matters because the rest of the codegen consumes the master RNG for codec sub-decisions, junk gadget choices, and handler temporary permutations: if an obfuscation pass at zero density were to draw from the master RNG just to decide that no transformation should be inserted, it would shift every downstream decision and produce a different output even when the pass nominally contributed nothing. Sub-RNG derivation prevents that, and lets the operator independently enable, disable, or scale either pass without perturbing the other axes of variance.

Dead-IR injection. The first pass walks the IR program at a configurable density, defaulting to 20% of inter-instruction gaps, and inserts an IMM instruction loading a random 64-bit value into one of the temporary registers Tmp2 or Tmp3 at each chosen gap. Tmp2 and Tmp3 are reserved by convention from lifter consumption: only Tmp0 and Tmp1 are read by lifter-emitted instructions, so writes to Tmp2 and Tmp3 are guaranteed dead. The IMM opcode produces no architectural side effect other than the slot write and, importantly, does not touch flags, so it can sit between any two flag-dependent instructions without disturbing their relationship. The pass skips synthetic blocks whose start_va sentinel indicates that the block was created by out-of-range jcc synthesis rather than by lifting real input bytes, because inflating those single-instruction native escape blocks would yield no analyst-noise gain in exchange for measurable bytecode growth.

Opaque predicates via block splitting. The second pass introduces conditional branches whose semantics make them deterministically never taken at runtime, but whose presence forces a static analyzer to reason about the targets they reference. At a configurable density, defaulting to 25% of blocks, the pass selects a block and splits it: the block's original IR is moved into a fresh synthetic tail block, and the original block identifier is repurposed for a prefix block containing three IR instructions, namely an IMM that loads zero into Tmp3, a TEST of Tmp3 against itself, and a BR_CC NZ whose target is a randomly chosen real block elsewhere in the program. At runtime the zero load forces the zero flag of the TEST to one, the not-zero conditional fails, and the branch is never taken; control instead falls through into the synthetic tail block that holds the original semantics.

The pass splits the block rather than inserting the predicate inline within the existing block because the BR_CC handler unconditionally invokes cipher_reset on both the taken and not-taken edges. The cipher reset is correct in the unmodified runtime, where every BR_CC is a block terminator and every target, taken or not, lands at a fresh block whose encoded bytecode begins with cipher_state at cipher_init. An inline mid-block BR_CC would reset the cipher on its not-taken edge, and the remaining instructions of the same block would then be decoded under the wrong cipher schedule, producing garbage opcodes that decrypt to handlers the encoder never emitted. Block splitting preserves the invariant: the prefix block is itself terminated by BR_CC, the synthetic tail block is a fresh block whose encoder also resets the cipher at its start, and both paths from the BR_CC arrive at fresh-block boundaries.



A handful of edge cases require care. The pass skips blocks whose first IR instruction reads flags, such as BR_CC, SETCC, or CMOVcc lifted to SETCC plus ZEXT, because the inserted TEST would clobber the flag state those readers depend on. The target of the never-taken branch is drawn from a snapshot of the real block identifier list captured before any splits begin, so newly synthesized tail blocks do not become predicate targets; that case would be confusing in static analysis output even though it would still be runtime-correct.

Four-Component Immediate Encoding and Codec Parity

Immediate constants present a particular problem for byte-level diversity. A literal value such as 0x4D5A0001 occurs verbatim in any naive encoding of the IR program, and a static scanner that watches for that specific constant finds it directly. The codec for IMM-like operations addresses this by encoding each immediate value as a four-component expression whose evaluation at runtime reconstructs the original value, but whose component bytes are pseudorandom and differ across both seeds and IMM sites.

Construction. Let v be the literal value being encoded and N the operand width in bits. The

codec draws three values a, b, and c uniformly at random across the unsigned N-bit range and computes a fourth value d such that the runtime recombination yields v. Specifically, d is set to a plus the exclusive-or of b and c, minus v, taken modulo 2 to the N. The encoder emits the four components a, b, c, and d in succession to the bytecode stream. At runtime the corresponding handler fetches the same four components in order and computes v back as a plus the exclusive-or of b and c, minus d, taken modulo 2 to the N. The construction is a small mixed Boolean-arithmetic identity[11]: the operations ADD, SUB, and XOR all distribute cleanly through arithmetic modulo 2 to the N, so the recombination is exact for any operand width, and the codec emits identical machine code shape on x86 and x64 even though the underlying register width differs.

Encoder side:

v = literal value, N = width in bits
draw a, b, c uniformly from 0 up to $2^N - 1$
d := a + b XOR c - v, taken modulo 2^N
emit bytes of a, b, c, d in order, each xored through the running cipher state

Decoder side, inside the IMM handler:

fetch a' through stream cipher
fetch b' through stream cipher
fetch c' through stream cipher
fetch d' through stream cipher
compute v' := a' + b' XOR c' - d', taken modulo 2^N
write v' into the destination slot

The salt fed into the per-IMM-site PRNG mixes the global cipher state with the current bytecode position, which ensures that two identical immediate values at different IMM sites within the same build encode to different component byte sequences. Two IMM sites holding the same value across two seeds also differ because the global cipher state component of the salt differs across seeds.

Codec parity. The codec interface in mkPIVM is structured so that each codec family defines both an encoder routine, called at build time to emit the operand bytes for a given IR operation, and a handler emitter, called at codegen time to emit the runtime machine code that fetches those same bytes back. Both sides operate over a stream cipher whose state advances exactly once per byte fetched. A correctness invariant of the system is therefore that the encoder and the handler agree on the byte count for every operand variant of every codec family. If the encoder emits five bytes for a given operand variant but the handler fetches only four, the cipher state at the next instruction will be one fetch step behind the encoder's expectation, and from that point on every decrypted byte will be wrong. The dispatcher then jumps to handlers that the encoder never emitted, the wrong handler fetches a different number of bytes, the drift propagates, and the runtime executes a chain of garbage opcodes that may or may not crash quickly, depending on what the garbage decrypts to.

The failure mode is subtle in practice because it presents as a runtime crash arbitrarily far downstream of the actual mismatch, often inside the body of a handler that has no defect of its own. The architectural defense against the failure mode is to keep the encoder and the handler emitter paired in the source code, to express the byte count as a shared constant between them, and to exercise both sides on a synthetic test corpus that touches every operand variant of every codec. The constant operand encoding described above interacts with this invariant directly: the encoder must emit exactly four width-N integers, and the handler emitter must fetch and combine exactly four width-N integers, for every IMM site of width N in every codec family that contains an IMM-like operation.

7. Runtime VM Architecture

The previous sections covered the lifter, the polymorphism axes, and the IR-level obfuscation and operand encoding. This section describes what the emitted blob actually does at

runtime: the in-memory state structure it maintains, the prologue and state-init sequence that prepares the structure on first entry, the dispatcher loop that fetches and executes one IR instruction at a time, the native-escape paths through which control transfers from interpreted bytecode back to host machine code, and the runtime nonce mechanism that masks the in-memory handler table per process.

VMState, Prologue, Dispatcher

VMState. The runtime maintains a single contiguous structure, the VMState, on the host stack of the thread that invoked the blob. The structure holds the virtualized values of architectural registers and lifter temporaries in the register slot region described in section 5, a scratch area named cipher_extra that holds bookkeeping for the stream cipher and the runtime nonce, a shadow stack used to discipline call and return semantics across the native-escape boundary, and an inverse substitution box for the cipher families that need one. All offsets within the VMState are computed at codegen time from the per-build VMConfig and are constant for a given blob.

The cipher_extra region uses fixed sub-offsets that the codegen consults when emitting helper code. The build-time initial cipher state cipher_init lives at offset 48 from the cipher_extra base. The runtime nonce derived per process lives at offset 80. The pre-decrypt save slots used when the lazy data-island decrypt of pack mode runs live at offsets 88 and 96. The inverse substitution box, when applicable, starts at offset 256 and occupies 256 bytes.

VMState layout, per build, codegen time:

offset 0 to reg_count*8 – 1
: **seed-permuted register slots holding AX/CX/DX/BX/SP/BP/SI/DI, R8..R15 on x64, Tmp0..Tmp3, plus internal slots**

cipher_extra base + 48
: **cipher_init, the build-time fixed initial stream cipher state**

cipher_extra base + 80
: **runtime_nonce, derived per process at state-init time**

cipher_extra base + 88
: **pre_decrypt_ip save slot, used by pack mode**

cipher_extra base + 96
: **pre_decrypt_cs save slot, used by pack mode**

cipher_extra base + 256
: **sbox_inv table, 256 bytes, populated only for the SBoxAdd cipher family**

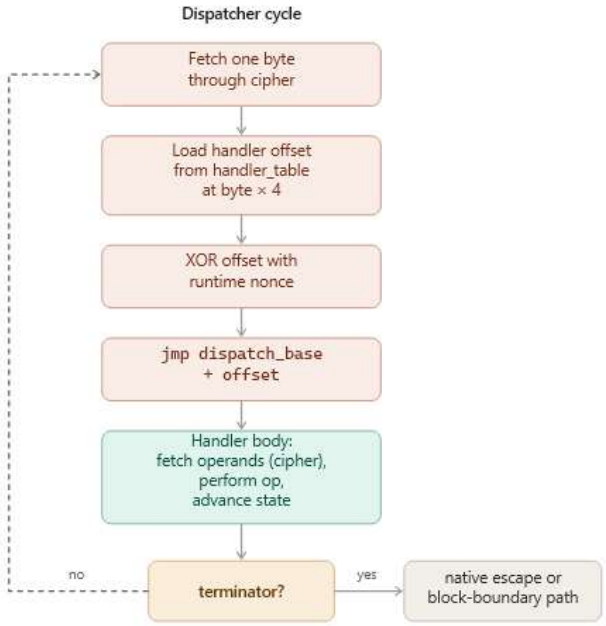
shadow stack region
: **VM_RSP-relative, addressed top down**

Prologue. The blob entry point begins with a prologue that establishes the host stack frame and locates the VMState in memory. The prologue pushes the host non-volatile registers in a seed-shuffled order onto the host stack so that they will be restored in matching order by the exit handler, pushes the flags register, allocates a frame for the VMState through a sub-rsp or sub-esp adjustment, and computes the state_ptr through whichever self-locate strategy was chosen for this build per section 5. After state_ptr is established, the prologue branches to the state-init body if the build-time-baked init_flag byte indicates that this is the first invocation since the blob was loaded into the host process. On subsequent invocations within the same process, the init_flag is set and the state-init body is bypassed.

State init. The state-init body performs the one-time setup that converts the as-emitted, encrypted-at-rest layout of the blob into a runnable interpreter. The architectural register slots are zeroed by xor-source instructions whose source register is itself a per-seed choice from the available volatile pool. The cipher state register is initialized to cipher_init, and cipher_init is also stored at cipher_extra+48 so that handlers performing cipher_reset can re-read it without recomputing. The inverse

substitution box is copied byte by byte from its location within the blob to its runtime offset at cipher_extra+256. A runtime nonce is derived by reading the host CPU timestamp counter via rdtsc, exclusive-or-ing it with cipher_init, and storing the result at cipher_extra+80. The data island, the block table, and the handler table are then decrypted in place through repeated calls to the cipher fetch operation. After the three regions are plaintext, the runtime nonce is xored into each entry of the now-plaintext handler table, so that the table in memory is masked by a value the dispatcher will reverse on every lookup. Finally, the init_flag byte is set to 1.

Dispatcher. With state init complete, control reaches the dispatcher tail. The dispatcher fetches one byte from the bytecode pointer through the stream cipher, advancing the cipher state by one step. It uses that byte as an index into the masked handler table, loads the four-byte signed handler offset stored there, xors that offset against the runtime nonce to unmask it, sign-extends to a 64-bit value on x64 or a 32-bit value on x86, adds the result to the dispatch base, and jumps to the resulting handler address. The handler body performs its semantic work, advances the cipher state by however many operand bytes it consumes, and either falls through to the next instruction under the threaded dispatcher shape or jumps back to the dispatcher tail under the central dispatcher shape, as described in section 5.



Native-Escape Paths and Shadow Stack Discipline

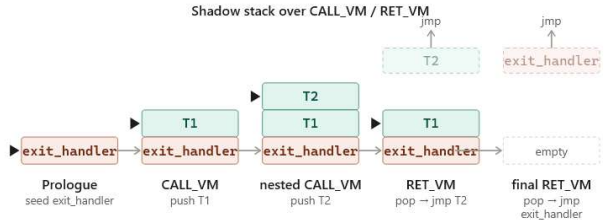
A virtualized payload cannot remain inside the dispatch loop forever; sooner or later the lifted code calls an external API, invokes an unresolved-at-build-time syscall, or returns to a caller outside the lifted region. mkPIVM provides three native-escape primitives that handle the boundary between interpreted bytecode and native execution: JMP_NATIVE, CALL_NATIVE, and RET_VM.

Marshalling. All three primitives perform the same architectural state transfer on the way out: the virtualized values in the VMState's register slots are loaded into the corresponding host registers, the host stack pointer is set to VM_RSP, which addresses the top of the VMState's shadow stack, and control jumps to the resolved native target. The shadow stack is what makes the discipline self-consistent: native code that runs on top of VM_RSP sees a stack that the VM controls, so when the native code eventually returns it pops from a location that the VM expects to find populated.

JMP_NATIVE. The unconditional native escape is used for control transfers that the lifted code expresses as JMP, plus the out-of-range jcc synthesis from section 4. The handler marshals state and jumps. The host stack pointer for the native target is set to the current VM_RSP, so the native code runs with whatever the shadow stack already holds. A handler-internal subtle point is that the marshalled jump is implemented as `jmp` through a register or memory operand rather than as `call`. Once host RSP has been set to VM_RSP, a CPU `call` instruction would push the eight-byte return address onto the shadow stack at an offset that would mis-align it with the rest of the shadow-stack discipline. Using `jmp` avoids the shadow-stack alignment hazard, at the cost that no automatic return is inserted; if the native code returns via `ret` it pops from VM_RSP exactly as the handler arranged.

CALL_NATIVE. The call-style native escape is used for calls to resolved API addresses and for in-blob native targets. Before marshalling, the handler pushes a trampoline address onto VM_RSP. The trampoline is a small piece of stub code emitted alongside the dispatcher, one trampoline per call site or per call site class. Its job is to receive control when the native callee returns and to resume VM dispatch from where the call originated. After the trampoline push, the handler marshals state and jumps to the native target. The native callee runs to its `ret`, pops the trampoline address from VM_RSP, and lands in the trampoline body, which moves control back into VM dispatch and reuses the cipher_reset path so that the post-return decoder state matches what the encoder produced for that resume point.

RET_VM. The return-from-VM primitive is the dual of CALL_NATIVE: it pops the top of the shadow stack into a scratch register, marshals state, and jumps to the popped address. If the original lifted code performed CALL_VM, which pushed a trampoline address onto VM_RSP as part of the call, then RET_VM pops that trampoline address and lands back in VM dispatch at the call's continuation. If the shadow stack is at its bottom-most slot when RET_VM fires, the popped address is the exit_handler that the prologue seeded there at state-init time. Control therefore lands in the exit handler, which performs the symmetric counterpart to the prologue: it restores the non-volatile registers in the inverse of the seed-shuffled order they were pushed in, restores the host flags, and returns to the caller of the blob's entry point.



The exit_handler seed at the bottom of the shadow stack is essential. Shellcodes whose top-level function ends with a real `ret` instruction would otherwise pop garbage and jump into a random byte address. With the seed in place, that final `ret` unwinds cleanly through the exit handler and returns to whatever loader code invoked the blob.

A range-mode subtlety arises when a JMP_NATIVE fires mid-execution from inside a lifted range, escaping to a target outside the range that the lifted code expected to reach by falling through into native bytes. The marshalled jump leaves the lifted region, but because the lifted region was entered via a five-byte jmp patch with the original caller's return address still on the host stack, a naive marshalling would either overwrite or fail to consume that retaddr correctly. The runtime resolves this by overwriting the caller's return address slot on the host stack with the JMP_NATIVE's target, then routing control through the exit handler's ret, which lands at the rewritten retaddr. The

mechanism is correct for the common case of a never-returning native target. Test builds use an int3 marker value in the retaddr slot to detect cases where the assumption is violated, such as a target that returns through the modified slot back into the wrapper.

Runtime Nonce Masking of the Handler Table

The handler table is a 256-entry array of four-byte signed offsets that map opcode bytes to the absolute address of their handlers relative to the dispatch base. Within a single process the table is a flat array of 1024 bytes that the dispatcher consults on every fetched opcode. Without further protection, the table is a strong signature target: any two processes loading the same blob would hold byte-identical handler tables at the same VMState offset, so a cross-process memory scanner that had a sample of the table from one process could match it directly against the address space of any other process running the same blob.

The runtime nonce masking mechanism breaks this match. During state-init the runtime executes rdtsc, which reads the host CPU's timestamp counter into the EDX:EAX register pair, xors the resulting 64-bit value against cipher_init, and stores the result at cipher_extra+80 as the per-process runtime nonce. After the handler table has been decrypted in place into its plaintext form, the runtime walks the table and xors each four-byte entry against the low 32 bits of the runtime nonce, producing a masked in-memory representation. The plaintext table is therefore never resident in memory in plaintext form: it is encrypted at rest during emission, decrypted in place during state-init, and immediately remasked with the nonce.

The dispatcher reverses the mask on every lookup. The masked four-byte handler offset is loaded from the table, xored against the runtime nonce stored at cipher_extra+80, sign-extended to the host pointer width, added to the dispatch base, and jumped to. The extra xor adds one instruction to the dispatcher hot path and zero new memory accesses, since the nonce is held adjacent to other state-init data that the dispatcher already touches.

Two consequences follow. First, two processes loading the same blob hold byte-different handler tables, because rdtsc returns different values in different processes and at different points in time. A cross-process scanner that has memorized the table from one process finds no match in the next, and a generic signature on the table would have to match the masked form, which depends on a per-process value the scanner cannot predict. Second, a single-process snapshot taken at any one time is also not transferable across that same process's lifetime: the nonce is fixed only after state-init, but a future build that re-enters state-init through a re-load would draw a different nonce, so the static signature target gains no leverage even from repeated sampling of a single host across runs.

8. Modes

Section 3 introduced the five operating modes at the level of the pipeline and the runtime data flow. This section describes each mode in detail, covering the implementation choices that distinguish it from the others, the build-time mechanics, the runtime invariants, and the operational tradeoffs.

Default Virtualization

Default virtualization lifts the entire input shellcode through the pipeline of section 4, applies the obfuscation passes of section 6.1, encodes the IR through the codec system of section 6.2, and emits a standalone position-independent blob. The output's entry point at byte 0 of the blob is the wrapper VM's prologue.

Execution remains inside the dispatcher loop from that point until either the lifted code performs a native escape to an external API or to its own native bytes, or RET_VM unwinds the shadow stack down to the exit_handler seed and returns to the caller.

The output blob layout in default mode is a single contiguous region. The wrapper VM, including the prologue, the state-init body, the dispatcher tail, the handlers, the per-call trampolines, and the exit handler, occupies the first part of the blob. The inverse substitution box follows, used only by the SBoxAdd cipher family. After that come the encrypted bytecode, the encrypted data island, and the encrypted block table, in that order. The encryption of each region uses the same per-build cipher family but resets the cipher state at the start of each basic block within the bytecode, so the decoder must reset in matching positions, as described in section 5.

Two side data structures support runtime semantics. The data island holds bytes that the original shellcode referenced through LEA addresses but that the CFG recovery classified as not belonging to any code block: string literals, lookup tables, jump tables, and the bytes that the shellcode itself reads through PEB walks and symbol hash comparisons. The block table maps the original virtual address offsets of the start of each block to the offset of the encoded bytecode that represents that block; it is consulted by RET_VM when the popped trampoline address falls inside the blob, allowing the call-return semantics to resume at the correct bytecode offset.

The default-mode blob is self-contained. The host loader allocates an RWX page, writes the blob in, and calls the entry point. No imports, no runtime resolver, no auxiliary section is required. The blob carries all the state it needs for the runtime to bootstrap, and the shellcode is responsible for whatever external resolution its execution requires.

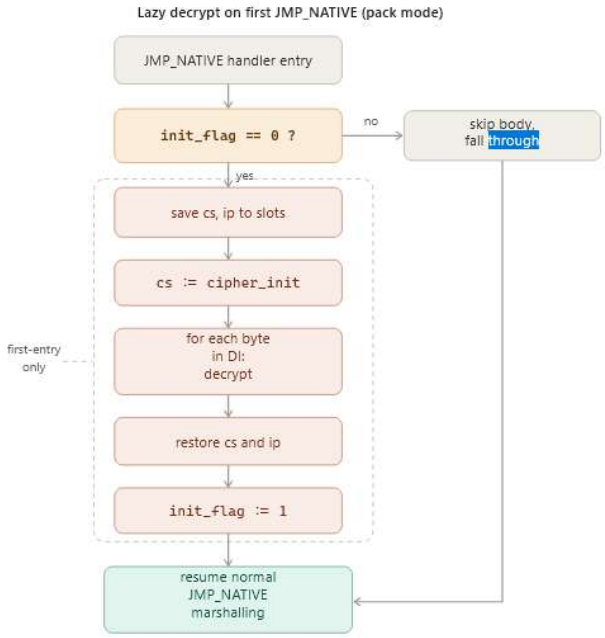
Packer Mode and Lazy Data-Island Decryption

Packer mode is the dual of default virtualization. The lifter does not run at all. Instead, the packager constructs a one-instruction IR program whose sole instruction is a synthetic JMP_NATIVE with an immediate operand of zero and an operand width of Q, the full pointer width of the target architecture. The original shellcode bytes are placed verbatim into the data island, encrypted in place at build time with the same cipher family that encrypts the rest of the blob. The wrapper VM is then assembled around this single-instruction program exactly as it would be assembled around any other IR program.

The runtime story for packer mode differs from default in one important way. The data island contains, after decryption, the original shellcode that the operator wants to execute, and this shellcode will be reached natively rather than through any VM dispatch. The encryption of the data island at rest is therefore a meaningful defensive layer at the static signature level, and the cost of decrypting it on entry to the wrapper is repaid only if the wrapper actually fires the synthetic JMP_NATIVE.

The naive approach, decrypting the data island eagerly in the state-init body alongside the bytecode and the handler table, was rejected for packer mode because the data island in pack mode is large compared to the rest of the blob, and decryption of large regions eagerly imposes a startup cost that scales with the input shellcode size rather than with the wrapper size. Instead, the data island decrypt is deferred to the first invocation of the JMP_NATIVE handler. The state-init body is gated to skip the data island decrypt when the build-time flag pack_mode is

true. When the wrapper first reaches its single JMP_NATIVE instruction, the handler checks an init flag byte stored alongside the cipher state. If the flag is zero, the handler saves the current cipher state and the bytecode instruction pointer into the pre-decrypt save slots at cipher_extra+88 and cipher_extra+96, resets the cipher state to cipher_init, walks the data island decrypting it byte by byte, restores the saved cipher state and instruction pointer from the save slots, sets the init flag to 1, and falls through into the normal JMP_NATIVE marshalling path. On subsequent invocations of the same handler within the same process, the init flag is set, and the decrypt body is skipped.



The single-fire structure of pack mode means that this lazy decrypt has no analogue in default or hybrid mode, where the lifted code uses LOAD and STORE against data island bytes mid-execution and requires them to be plaintext from the very first dispatcher fetch.

Packer mode preserves the full per-seed polymorphism of the wrapper VM. The same eight axes vary, the wrapper is byte-different across seeds, and the encrypted data island is salted with the per-seed cipher initial state. What pack mode loses, relative to default virtualization, is per-instruction obfuscation: once the wrapper hands control to the decrypted shellcode, the shellcode bytes execute natively, and a runtime memory dump will recover them in plaintext. The mode is therefore the right shape for shellcodes that are too large to lift, that contain mnemonic categories the lifter does not cover, or that the operator wants to keep native for compatibility reasons.

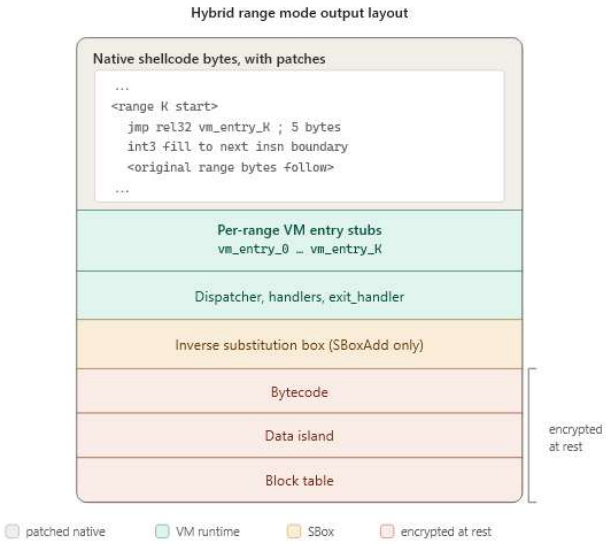
Hybrid Range Mode and Patch-Site Discipline

Hybrid range mode is the intermediate point between default and pack. The operator chooses byte ranges of the input shellcode that should be virtualized; everything outside those ranges remains native. The mode is the right shape when only a subset of the shellcode is amenable to lifting, when the operator wants per-instruction obfuscation on selected critical sections, or when full virtualization is undesirable for performance reasons.

Build time. The packager parses the --ranges flag into a list of half-open byte intervals. The CFG builder is configured with this list and restricts its recursive descent to blocks whose start virtual address is inside any range. Blocks outside the ranges are not lifted. Branches and calls inside lifted blocks whose targets leave the range become JMP_NATIVE or CALL_NATIVE respectively, as described in section 4. The bytecode is encoded normally, but the surrounding output layout differs from default mode.

The output begins with a verbatim copy of the original native shellcode. At each range start, the packager overwrites the first five bytes of the range with a jmp rel32 to the corresponding per-range VM entry stub, which is appended after the native region. The remaining bytes of the displaced run, those between the end of the five-

byte patch and the next instruction boundary in the original shellcode, are filled with int3, so that any control transfer that lands on a mid-range byte from outside terminates loudly rather than silently proceeding through corrupted bytes. The VM stub, the inverse substitution box, the bytecode, the data island, and the block table follow.



Per-range entry stubs. Each range has its own entry stub vm_entry_K that performs the prologue, state-init dispatch, and dispatcher entry for that particular range. The per-range arrangement is necessary because each entry needs to push and later pop the host non-volatile registers in the same seed-shuffled order: if the order differed across entries, the exit handler could not unwind correctly because it does not know which entry was used.

The NV-push order is cached as part of the VMConfig and used identically by every entry stub and by the exit handler. The init_flag byte is shared across all entries, so the first range entered triggers state-init and subsequent entries skip it. Both of these invariants are load-bearing: a multi-range build with inconsistent NV-push orders or inconsistent init gating produces undefined behavior the first time control crosses between ranges.

Patch-site discipline. The five-byte jmp patch overwrites whatever instructions were originally at the range start. The first byte of the range becomes 0xE9 and the next four are the rel32 displacement. The remaining bytes of any displaced original instruction that the five-byte patch partially overwrote are filled with int3. This means that external native code outside the range can only validly re-enter the range through the patched start byte; any control flow that lands on a mid-range byte from outside is undefined, and the int3 fill ensures it traps rather than proceeding silently. The scan-mode classifier discussed in section 3 enforces this property at scan time: ranges whose CFG analysis reveals external native branches into mid-range bytes are classified as near-miss rather than eligible.

Range-exit paths. Lifted code that branches to a target outside its range exits through a JMP_NATIVE or CALL_NATIVE, but the path differs slightly from default-mode native escape because the range entry stub on the host stack has a frame that needs to be unwound. The JMP_NATIVE handler in range mode therefore overwrites the caller's return address slot on the host stack with the target virtual address and then routes control through the exit_handler's ret, which lands at the rewritten retaddr after the NV-push order has been undone, as described in section 7.2.

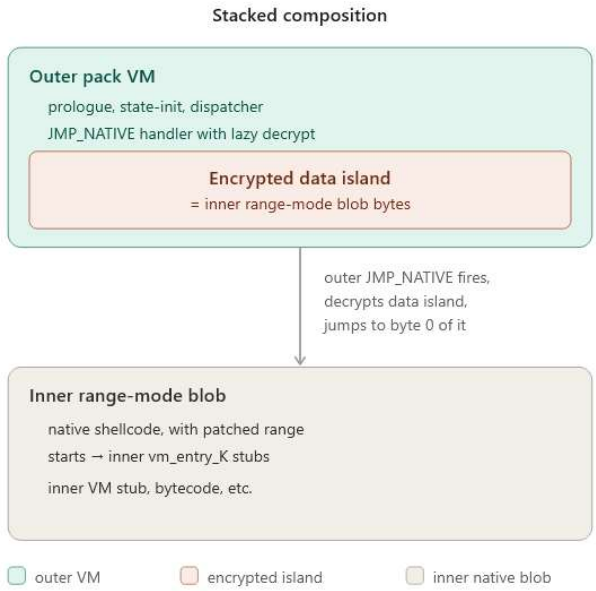
Coroutine ranges. Some shellcode functions return through tail-jmps rather than through a ret instruction; such ranges have no ret-terminated block. The scan classifier flags these as coroutine candidates. The runtime supports them through the same JMP_NATIVE retaddr-overwrite mechanism: the coroutine exits by jumping to a native target that the caller expected, and the host-side cleanup proceeds identically to the ret-terminated case.

Stacked Mode

Stacked mode composes packer mode around hybrid range mode. The packager invokes itself recursively in range-only mode on the input to produce an intermediate range-mode blob. That intermediate blob is then fed as the input to a second self-invocation in pack-only mode. The output is a packed PIC blob whose decrypted contents are themselves a range-mode PIC blob.

Two virtual machines therefore exist in the output, one nested inside the other. The outer pack VM is the one that the host loader's entry-point call lands in. The outer VM's dispatcher fetches the single synthetic JMP_NATIVE that pack mode emits, the JMP_NATIVE handler performs the lazy data island decrypt on the inner range-mode blob bytes, and the marshalled jump transfers control to byte 0 of the now-plaintext inner blob. From that point on the inner blob runs exactly as a standalone range-mode build would: the inner native shellcode prologue executes, the patched range starts redirect into the inner range entry stubs, and the inner dispatcher loop drives execution through the lifted ranges.

The two virtual machines share no state. They have distinct VMState frames on the host stack, distinct cipher state schedules, distinct handler tables, distinct dispatch bases, and distinct shadow stacks. The outer VM exists solely to gate the inner VM behind a decryption boundary. The inner VM does not know that an outer VM exists, and the outer VM does not know that the data island it decrypts and jumps into contains another VM.



The composition is clean because the inner blob is by construction a self-contained position-independent region. Its execution semantics depend only on the absolute address at which it starts running, and the outer pack VM places it at a known address before transferring control. Any range-mode blob can be wrapped by pack mode, and any pack-mode blob can wrap a range-mode blob; the operator chooses based on whether per-instruction virtualization of selected regions plus an outer encryption layer is the right tradeoff for the payload.

PE Detour, Threaded and Inline Sub-Modes

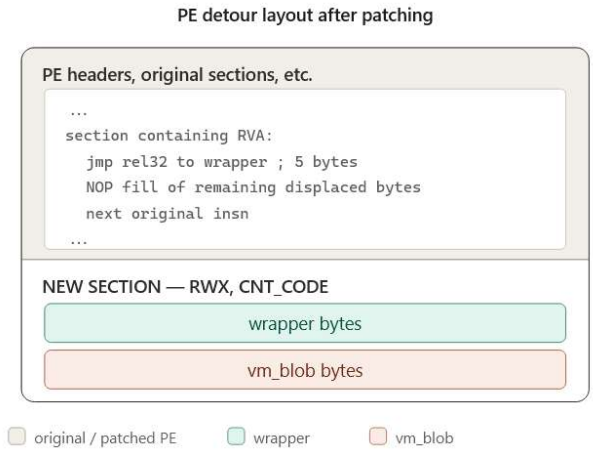
PE detour mode does not produce a raw position-independent blob. It produces a patched Portable Executable[2]. The input to detour mode is the raw shellcode that the operator wants to embed, conventionally a previously emitted mkPIVM blob but any position-independent bytes will work, together with a target executable and a relative virtual address inside that executable at which to install the patch.

Common build steps. The detour tool reads the target PE bytes, parses the DOS header, NT headers, sections, optional header, and base

relocation directory. It checks that the PE's architecture matches the blob's architecture: PE32+ for x64 blobs, PE32 for x86 blobs. It locates the chosen relative virtual address inside an executable section. It disassembles forward from the relative virtual address using Zydis[6], accumulating instructions until the cumulative length reaches or exceeds five bytes. This is the displaced run that the five-byte jmp patch will overwrite. The tool then validates the displaced run against two unsafe properties: no instruction in the displaced run may have a RIP-relative memory operand, and no instruction in the displaced run may be a rel32 branch. Both properties would break when the displaced instructions are moved to a new location, because the relative offsets they encode are anchored to their original position. If either property is found, the detour aborts, and the operator must pick a different relative virtual address.

If the displaced run is safe to relocate, the tool emits a wrapper. The exact wrapper bytes depend on the sub-mode; both sub-modes share the same overall shape: save host state, transfer to the blob, restore host state, execute the displaced original bytes verbatim, jump back to the byte after the patch.

The wrapper is concatenated with the blob bytes, and the combined section is appended to the PE as a new section with RWX memory protection and the CNT_CODE characteristic flag. The section table is extended by one entry, NumberOfSections is incremented, SizeOfImage is bumped accordingly, and the optional header's CheckSum field is zeroed. The five-byte jmp rel32 from the chosen relative virtual address into the new section is written into the original section's bytes, and any bytes of the displaced run beyond the first five are NOP-filled.



Threaded sub-mode. The threaded variant runs the blob in a separate worker thread so that the host's main thread is unblocked. The wrapper preserves host state, sets up arguments for a CreateThread call, calls kernel32!CreateThread through the host PE's import address table entry for that function, restores host state, executes the displaced bytes, and jumps back. The CreateThread address is resolved at build time by scanning the host's import directory for kernel32!CreateThread. If the host does not import the function, the tool refuses and suggests inline mode instead. On x64 the call into CreateThread uses a `call qword ptr[rip + iat_disp32]` form, where the iat_disp32 is computed at build time to point from the wrapper's call site at the IAT entry. On x86 the call uses `call dword ptr[iat_va]`, where iat_va is the absolute address of the IAT entry. Because the x86 form contains absolute addresses, the tool emits matching IMAGE_BASE_RELOCATION entries with HIGHLOW type into a combined base relocation table that replaces the host's original BASERELOC directory.

Inline sub-mode. The inline variant calls the blob synchronously from the wrapper, blocking the host's main thread until the blob returns or terminates. The wrapper preserves host state, calls the blob through a `call rel32` instruction whose relative displacement is computed at

build time, restores host state, executes the displaced bytes, and jumps back. No IAT scan is required because no external API is called, no x86 base relocations are needed because all references in the wrapper are rel32-relative, and the variant works against host PEs that lack the imports needed for the threaded sub-mode.

The two sub-modes are operationally different but architecturally close: the wrapper is the same shape, the displaced-bytes preservation discipline is the same, and the post-displaced-bytes jump back to the byte after the patch is the same. The choice between them is driven by whether the blob's execution should block the host's main thread, and by whether the host PE's import table contains kernel32!CreateThread to enable the threaded path.

9. Discussion and Related Work

Limitations and Deliberate Tradeoffs

The architecture described in this paper is the product of a series of decisions, several of which trade defensive scope for operational pragmatism. This subsection enumerates the most important of those tradeoffs along with the limitations they introduce.

Runtime memory protection. The output blob requires writable and executable pages because the prologue decrypts the encrypted-at-rest regions in place into the blob's own backing memory. The decrypt body could in principle be rewritten to allocate fresh writable pages, copy the encrypted blob to them, decrypt there, mark them executable, and jump in, which would let the original blob live in pages without write permission. The reason this design was rejected is that the rewrite would require the wrapper to perform a PEB walk to resolve VirtualAlloc, call VirtualAlloc, copy several kilobytes, and call VirtualProtect, all before reaching the first useful instruction. The resulting wrapper is itself a strong static signature target precisely because PEB-walking allocators have well-documented instruction patterns. The conclusion is that RWX-at-runtime is a worse signal in the abstract but a more defensible signal in practice than the wrapper code an RX-only version would require.

Lifter coverage. The lifter covers the working set of mnemonics observed in shellcodes and stagers but does not cover every x86 or x64 instruction. SSE and AVX register moves and arithmetic, atomic operations including CMPXCHG, privileged instructions such as RDMSR and WRMSR, and a handful of less-common control flow primitives are not yet lifted. Shellcodes that depend on those mnemonics either fail to lift in default mode or produce a build error. Pack mode is the operational workaround for such shellcodes, since the lifter does not run there at all, but the operator forfeits per-instruction virtualization for the affected payload.

Range mode applicability. Range mode lifts only the operator-chosen byte ranges and represents the rest of the input as native bytes. The mode works well for self-contained functions such as the entry point of a stager or the body of a well-isolated leaf. It works less well for helper functions whose semantics depend on caller-supplied register state that the lifted entry stub does not know how to materialize. Cobalt Strike stager helpers are the canonical example: the helper depends on a specific register state established by its caller, and the range-mode runner does not provide that state, so the helper executes against garbage arguments. Default virtualization or pack mode are the correct routes for such payloads.

Authenticode invalidation. PE detour mode invalidates any Authenticode signature on the target executable, because the section table, the section bytes, and the optional header checksum all change. The detour tool zeros the checksum and does not attempt to re-sign the patched

output. Code-signing-aware loaders that verify the signature will refuse the patched binary; this is by design.

Anti-debug and anti-virtualization. The output blob does not check for the presence of debuggers, does not detect emulation environments, and does not change its behavior when running under instrumentation. Adding such checks would itself create signature targets, since the instruction patterns of common anti-debug primitives are well documented in detection databases, and would offer at best a temporary advantage against an analyst with the time and motivation to remove them.

Userland API hooking. The blob does not interfere with userland hooks installed on ntdll or other Windows API surfaces. If the input shellcode wants to bypass such hooks, it must do so through its own mechanisms, for example by performing direct or indirect syscalls[3]. The architecture is transparent to that choice and imposes no restriction on the inner payload's API invocation style.

Kernel-mode scope. mkPIVM operates entirely in userland. Kernel callbacks, Early Launch Anti-Malware enforcement, PatchGuard interactions, and other kernel-surface defenses are out of scope and unaffected by anything the tool does or produces.

Output size. The emitted blob is several times larger than the input shellcode. Default-mode output for a small MessageBox demo lands in the tens of kilobytes, dominated by the codec handlers and the dispatcher machinery; the bytecode and tables grow proportionally to input size. Pack mode is the most compact relative to input, since the wrapper has fixed size and the input goes into the data island verbatim. The size cost is the per-build polymorphism: a fixed wrapper paired with a fixed cipher would compress to almost nothing, but its byte image would not vary across builds.

Dispatch performance. Every virtualized native instruction becomes several host instructions: a fetch through the cipher, a handler-table lookup with nonce unmask, an indirect jump, the handler body, and the operand fetches. The dispatch overhead is on the order of tens of host cycles per virtualized instruction. For control-bound shellcodes the slowdown is in the low single-digit factor; for arithmetic-bound code it can be larger. Pack mode and range mode exist as operational answers when this overhead matters.

Positioning Against Commercial and Academic Prior Art

mkPIVM occupies a region of the design space that, to the best of our knowledge, no public tool currently ships exactly. Several adjacent points in that space are worth naming explicitly.

Commercial code virtualizers. VMProtect, Themida, and Code Virtualizer are the established commercial implementations of the lift-to-VM technique for software protection. Their input is a linked Windows Portable Executable[2] with imports, sections, and loader metadata; their output is itself a PE that depends on a fully formed loader environment. The technique class is the same as the lift-to-VM half of mkPIVM, but the deployment surface is fundamentally different: a PE-targeted virtualizer cannot be applied directly to raw shellcode, and a shellcode-targeted virtualizer cannot directly leverage the import resolution, section management, and runtime resolver scaffolding that those tools rely on. The two systems coexist as solutions to different deployment problems within the same broad technique class.

Public polymorphic encoders. shikata_ga_nai[5] and the broader family of msfvenom encoders apply a per-build XOR-style transformation to the shellcode body, prefixed by a small decoder stub whose

instructions are dynamically reordered and register-substituted per build. The encoder layer is polymorphic, but the wrapper stub is structurally identical across builds and the underlying shellcode is recovered verbatim in memory before it executes. mkPIVM's pack mode is the closest analogue at the level of raw deployment shape, but its wrapper is itself polymorphic across all the axes described in section 5, and at the level of default virtualization its semantic transformation is qualitatively different: the original instructions never appear verbatim in memory because they are interpreted from bytecode.

Position-independent loaders. Donut and similar tools take an arbitrary executable as input and produce position-independent shellcode that loads and runs it from memory. The output is position-independent and signature-resistant relative to plain executables, but Donut is a loader-generator rather than a code-virtualizer: the wrapped executable is decompressed and executed natively after a fixed-shape stub runs. The shape of the stub is what end-point scanners eventually learn to recognize.

Historical polymorphism. Polymorphic decryptors in early viruses such as the 1260 virus and the Mutation Engine[4] established the technique class of per-build byte diversity at the wrapper level. Those engines varied the decryptor's instruction sequence rather than the encrypted payload's semantic representation. mkPIVM extends the lineage by treating both the wrapper and the encrypted instruction representation as independent axes of variance, and by varying the VM that interprets the encrypted representation rather than only the decryptor that recovers it.

Academic deobfuscation. Academic analysis of virtualization-based obfuscation, beginning with Rolles's WOOT 2009 paper on unpacking virtualization obfuscators[12], has shown that single-seed VM-based protection is amenable to systematic reverse engineering once the dispatch loop is identified and the handler semantics enumerated. mkPIVM's per-seed polymorphism is in part a response to that line of work: the recovery effort that an analyst expends against one captured sample does not generalize across samples of the same input because the cipher family, the opcode mapping, the register layout, the dispatcher topology, and the handler internals all differ between any two seeds. The architectural assumption is that the deobfuscation work required for a per-instance VM is the dominant analyst cost, and forcing that cost to be paid per sample is the principal defensive value of the design.

9. Conclusion

This paper has introduced the concept of position-independent virtual machines, a class of process virtual machines whose entire image, comprising the dispatcher, the handler table, the encrypted bytecode, and the encrypted data island, is itself position-independent code that can be invoked from any loader that accepts native shellcode. The construct preserves the deployment surface of native shellcode while replacing the static byte image with a per-instance, per-seed virtual machine.

We have presented mkPIVM, an implementation of this concept for Windows x86 and x64. The tool lifts arbitrary input shellcode to a custom intermediate representation through a recursive descent disassembler[7] over Zydis[6], applies IR-level obfuscation including dead-instruction injection and opaque predicates via block splitting[1], encodes the result into per-seed virtual machine bytecode under one of four stream cipher families drawn from the established cryptographic literature[8][9][10], and emits a position-independent blob whose byte

image varies substantially between any two builds.

Eight axes of per-seed variance vary independently: the cipher family, the register slot permutation across the virtual machine state, the opcode-to-handler mapping, the dispatcher topology, the prologue self-locate strategy, the per-handler temporary register layout, the junk gadget density, and the runtime nonce that masks the in-memory handler table per process. The four-component mixed Boolean-arithmetic encoding[11] of immediate constants extends the variance into the operand stream. The orthogonality of the axes, enforced through sub-RNG isolation in the codegen, ensures that a static signature anchored on any one axis fails to match a build whose other axes have rotated.

Five operating modes cover the breadth of payload deployment shapes encountered in modern offensive tooling: full virtualization, packer mode for payloads too exotic to lift in full, hybrid range mode for selective virtualization of operator-chosen byte ranges, a stacked composition that pack-wraps a hybrid blob, and a PE detour mode that embeds a pre-built blob into an arbitrary Portable Executable[2] and patches a five-byte jump at an operator-chosen relative virtual address so that the virtual machine only fires when the host process reaches that point.

The design occupies a previously unfilled region of the established design space for offensive code protection. Commercial virtualizer-protectors operate at the PE level and cannot be applied to raw shellcode without scaffolding that their inputs and outputs are not built to accommodate. Public polymorphic encoders apply per-build transformations to the wrapped body but leave the wrapper structurally identical across builds, and the underlying shellcode is recovered verbatim in memory before it executes. Position-independent loaders address the deployment-shape question but do not virtualize the wrapped code. mkPIVM combines all three lines into a single tool whose input is raw position-independent shellcode and whose output is raw polymorphic position-independent virtualized shellcode.

Several directions remain open for future work. Lifter coverage for SSE, AVX, and atomic instructions would broaden the set of shellcodes amenable to default virtualization. An RX-only output mode, possibly through self-relocating decrypt-to-fresh-pages, would address the runtime memory-protection tradeoff for environments in which RWX is itself a strong signal. Additional IR obfuscation passes drawn from the wider taxonomy of obfuscating transformations[1] would deepen the analyst-cost penalty per seed. Architectures beyond x86 and x64, ARM64 in particular, would extend the work to mobile and embedded shellcode contexts. Each of these directions builds on the same architectural foundation: the polymorphic, position-independent virtual machine that, by construction, presents a different image to every static observer and a different memory layout to every running process.

References

[1] C. Collberg, C. Thomborson, and D. Low, "A Taxonomy of Obfuscating Transformations," Technical Report #148, Department of Computer Science, The University of Auckland, New Zealand, July 1997.[Online]. Available: https://researchspace.auckland.ac.nz/handle/2292/3491

[2] Microsoft, "PE Format," Win32 apps documentation.[Online]. Available: https://learn.microsoft.com/en-us/windows/win32/debug/pe-format

[3] Aleph One, "Smashing The Stack For Fun And Profit," Phrack Magazine, vol. 7, no. 49, file 14, November 1996.[Online]. Available: https://phrack.org/issues/49/14

[4] P. Szor, The Art of Computer Virus Research and Defense. Boston, MA: Addison-Wesley Professional, 2005. ISBN 0-321-30454-3.

[5] spoonm, "Polymorphic XOR Additive Feedback Encoder, shikata_ga_nai," Metasploit Framework.[Online]. Available: https://github.com/rapid7/metasploit-framework/blob/master/modules/encoders/x86/shikata_ga_nai.rb

[6] J. Krause and Florian Bernd, "Zydis: Fast and Lightweight x86/x86-64 Disassembler and Code Generation Library," zyantific.[Online]. Available: https://github.com/zyantific/zydis

[7] B. Schwarz, S. Debray, and G. Andrews, "Disassembly of Executable Code Revisited," in Proceedings of the Ninth Working Conference on Reverse Engineering, Richmond, VA, USA, October 2002, pp. 45 to 54. DOI: 10.1109/WCRE.2002.1173063.

[8] D. J. Bernstein, "ChaCha, a variant of Salsa20," Workshop Record of SASC 2008: The State of the Art of Stream Ciphers, January 2008.[Online]. Available: https://cr.yp.to/chacha/chacha-20080128.pdf

[9] D. H. Lehmer, "Mathematical Methods in Large-Scale Computing Units," in Proceedings of a Second Symposium on Large-Scale Digital Calculating Machinery, Cambridge, MA, USA, 1949, pp. 141 to 146.

[10] H. Feistel, "Cryptography and Computer Privacy," Scientific American, vol. 228, no. 5, pp. 15 to 23, May 1973.

[11] Y. Zhou, A. Main, Y. X. Gu, and H. Johnson, "Information Hiding in Software with Mixed Boolean-Arithmetic Transforms," in Proceedings of the 8th International Workshop on Information Security Applications, WISA 2007, Jeju Island, Korea, August 2007, vol. 4867 of Lecture Notes in Computer Science. Berlin, Germany: Springer, pp. 61 to 75.

[12] R. Rolles, "Unpacking Virtualization Obfuscators," in Proceedings of the 3rd USENIX Workshop on Offensive Technologies, WOOT 2009, Montreal, Canada, August 2009.[Online]. Available: https://www.usenix.org/legacy/event/woot09/tech/full_papers/rolles.pdf

Appendix A: Per-Seed Diff Examples

This appendix shows concrete byte-level differences between three builds of the same input shellcode under three different seeds. The input is a small x64 MessageBox demonstration shellcode of 433 bytes. The seeds are randomized. The output blob sizes are 21778, 22045, and 23316 bytes respectively; the difference in length comes from the per-seed choice of cipher family, dispatcher topology, junk gadget density, and operand encoding, all of which influence the encoded representation size.

Prologue Bytes

The first thirty-two bytes of each blob contain the wrapper VM's prologue: the host non-volatile register push sequence in the seed-shuffled order, a pushfq, optional junk gadgets, and the sub-rsp adjustment that allocates the VMState frame.

```
random seed #1, blob[0x00..0x1F]:
0x0000:  41 56 57 53 41 57 41 55 56 55 41 54 9c 0f 1f 44
0x0010:  00 00 4d 85 f6 90 0f 1f 40 00 48 81 ec 00 24 00

random seed #2, blob[0x00..0x1F]:
0x0000:  41 57 41 55 55 56 41 56 57 41 54 53 9c 0f 1f 44
0x0010:  00 00 90 48 81 ec d0 1d 00 00 48 8d b4 24 00 16

random seed #3, blob[0x00..0x1F]:
0x0000:  41 56 53 41 54 41 55 56 41 57 57 55 9c 0f 1f 44
0x0010:  00 00 66 90 4d 31 ff 48 81 ec 20 0f 00 00 48 8d
```

The first twelve bytes of each prologue are the host NV-register push sequence. Each push of a non-extended GPR is a single-byte opcode, and each push of an extended GPR is a two-byte opcode beginning with the REX.B prefix 0x41. Reading the three sequences:

```
random seed #1 order: r14 rdi rbx r15 r13 rsi rbp r12
random seed #2 order: r15 r13 rbp rsi r14 rdi r12 rbx
random seed #3 order: r14 rbx r12 r13 rsi r15 rdi rbp
```

These are three independent permutations of the same set of eight non-volatile GPRs. The exit handler reverses each push in matching order, so the property is invertible per build.

After the push sequence each prologue emits pushfq, encoded as 0x9C, then begins to differ further at byte 0x0D. The #1 build inserts a five-byte NOP followed by a `test r14, r14` and a two-byte NOP, the #2 build inserts a single-byte NOP and proceeds directly to a sub-rsp with a 0x1DD0 displacement, and the #3 build inserts a `nop` then `xor r15d, r15d` before its sub-rsp with a 0x0F20 displacement. The differences in displacement reflect different per-seed VMState sizes that result from the cipher family choice and the slot-permutation layout.

Deep Blob Bytes

The bytes deep inside the blob, beyond the prologue and state-init code, are where the handler tables, the encrypted bytecode, the data island, and the block table live. Two of those regions are encrypted at rest under a per-seed stream cipher and therefore appear as uniform random byte sequences with no structural pattern. The handler bodies themselves are not encrypted, but they vary because each handler is emitted

with a different host-register layout under the per-handler temporary permutation, and the handler addresses are placed at different absolute offsets depending on the junk gadget density between them.

```
random seed #1, blob[0x1000..0x101F]:
0x1000:  ff c3 4c 03 af 88 01 00 00 49 c1 c5 0d 45 30 ee
0x1010:  4d 0f b6 f6 0f b6 13 48 ff c3 4c 03 af 88 01 00

random seed #2, blob[0x1000..0x101F]:
0x1000:  48 c1 c7 0d 41 30 fd 4d 0f b6 ed 47 8b 24 ae 44
0x1010:  33 a6 60 01 00 00 4d 63 e4 4f 8d 2c 26 41 ff e5

random seed #3, blob[0x1000..0x101F]:
0x1000:  01 00 00 48 03 ae f8 00 00 00 4e 8b 74 fe 08 48
0x1010:  63 c9 49 01 ce 49 8b 0e 44 0f b6 37 48 ff c7 41
```

At offset 0x1000 no two of the three blobs share any aligned byte position with the same value. The #1 and #2 blobs both contain instructions that load a state-relative address through 32-bit displacements with values 0x188 and 0x160 respectively, both of which encode the dispatch base offset within the VMState. The displacement values themselves are seed-specific because the cipher_extra region's position inside the VMState varies with the slot count, which is itself seed-dependent.

The IMM Operand Encoding

A single IR-level IMM operation that loads the immediate value 0x42 into a temporary slot encodes to four components of the modular identity in section 6.2. Each component is xored byte by byte through the stream cipher of the chosen cipher family. As a result, the four-byte sequence that materially encodes the value 0x42 differs between any two builds for two independent reasons. First, the values a, b, and c are drawn from a per-IMM-site sub-RNG salted with the current cipher state and the bytecode position, so they differ across seeds and across IMM sites. Second, the cipher itself differs in family and initial state, so even if a, b, c, and d were held constant across seeds, the encrypted bytes would still differ.

The combined effect is that no two builds of the same input share any contiguous substring of length greater than a small constant. Empirically the longest common substring across builds of the messagebox demonstration shellcode falls in the high tens to low hundreds of bytes, dominated by encoder-emitted boilerplate that does not depend on the seed: a few common multi-byte NOPs, the REX-prefixed push opcodes, and a few literal constants that appear in both builds at different positions but with identical byte patterns. The architectural choices documented in this paper are responsible for the byte-level divergence; the residual overlap is the noise floor below which the construction cannot fall without breaking the invariants the wrapper relies on.